

Lending Flow 360

Interview Master Pack (250 Questions + Answers)



Document by:

PST IT SOLUTIONS

**Main Branch: Bharat Bazar C-Wing First Floor
Beside Prozon Mall, Chatrapati Sambhajinagar
Contact No : 8446828029**

**Branch 2: Pt 79 , Second Floor,
Arya Nagar, Near Sugat Nagar Ring Road, Nagpur
Contact No : 8421128029**

Foundational interview questions and answers covering Apache Spark, Delta Lake, and Databricks concepts. These are ideal for building confidence in core data engineering topics and explaining your LendingFlow360 project.

Section 1: Basic Questions & Answers

◆ Apache Spark Basics

1. **What is Apache Spark?**
Spark is a distributed computing engine for big data processing. It supports in-memory computation and parallel processing across clusters.
2. **What are the main components of Spark?**
 - Spark Core
 - Spark SQL
 - Spark Streaming
 - MLlib (Machine Learning)
 - GraphX (Graph processing)
3. **What is an RDD?**
Resilient Distributed Dataset — a low-level abstraction for distributed data. Immutable and fault-tolerant.
4. **What is a DataFrame in Spark?**
A distributed collection of data organized into named columns. It's optimized and easier to use than RDDs.
5. **What is a Dataset?**
A strongly typed version of DataFrame available in Scala and Java. Combines benefits of RDDs and DataFrames.
6. **How does Spark achieve fault tolerance?**
Through lineage and DAGs (Directed Acyclic Graphs). Lost partitions are recomputed from original data.
7. **What is lazy evaluation in Spark?**
Transformations are not executed until an action (e.g., `collect`, `count`) is called.
8. **What is a transformation in Spark?**
An operation that returns a new RDD/DataFrame (e.g., `map`, `filter`, `select`).
9. **What is an action in Spark?**
An operation that triggers execution (e.g., `collect`, `count`, `show`).
10. **What is Spark SQL?**
A module for querying structured data using SQL syntax or DataFrame API.

◆ PySpark Essentials

1. **How do you read a CSV file in PySpark?**

```
df = spark.read.option("header", True).csv("path/to/file.csv")
```

2. **How do you write a DataFrame to Parquet?**

```
df.write.parquet("path/to/output")
```

3. **How do you filter rows in PySpark?**

```
df.filter(df["column"] > 100)
```

4. **How do you select specific columns?**

```
df.select("col1", "col2")
```

5. **How do you rename a column?**

```
df.withColumnRenamed("old_name", "new_name")
```

6. **How do you add a new column?**

```
df.withColumn("new_col", df["existing_col"] * 2)
```

7. **How do you drop duplicates?**

```
df.dropDuplicates()
```

8. **How do you join two DataFrames?**

```
df1.join(df2, on="id", how="left")
```

9. **How do you group and aggregate in PySpark?**

```
df.groupBy("category").agg(sum("amount"), avg("score"))
```

10. **How do you cache a DataFrame?**

```
df.cache()
```

◆ **Delta Lake Fundamentals**

1. **What is Delta Lake?**

A storage layer that brings ACID transactions, schema enforcement, and time travel to Spark.

2. **What file format does Delta Lake use?**

Parquet, with additional transaction logs.

3. **How do you create a Delta table?**

```
df.write.format("delta").saveAsTable("schema.table_name")
```

4. **How do you read a Delta table?**

```
spark.read.format("delta").table("schema.table_name")
```

5. **What is time travel in Delta Lake?**

Ability to query previous versions of a table using version number or timestamp.

6. **How do you update a Delta table?**

Using SQL:

```
UPDATE table_name SET col = value WHERE condition
```

7. **How do you delete from a Delta table?**

```
DELETE FROM table_name WHERE condition
```

8. **What is schema enforcement in Delta Lake?**

Prevents writing data that doesn't match the table schema.

9. **What is schema evolution?**

Allows adding new columns during write using `mergeSchema=True`.

10. **How do you merge data into a Delta table?**

Using `MERGE INTO` for upserts:

```
MERGE INTO target USING source ON condition WHEN MATCHED THEN UPDATE ...
```

◆ Databricks Platform Basics

1. **What is Databricks?**

A cloud-based platform for big data analytics built on Apache Spark. It supports notebooks, jobs, clusters, and governance.

2. **What is a Databricks notebook?**

An interactive workspace to write and execute Spark code in Python, SQL, Scala, etc.

3. **What is a cluster in Databricks?**

A set of virtual machines used to run Spark jobs. Can be interactive or job-based.

4. **What is auto-scaling in Databricks?**

Automatically adjusts the number of worker nodes based on workload.

5. **What is a job in Databricks?**

A scheduled or triggered execution of a notebook or workflow.

6. **How do you pass parameters to a notebook job?**

Using `dbutils.widgets`:

```
dbutils.widgets.text("biz_date", "") biz_date = dbutils.widgets.get("biz_date")
```

7. **What is Unity Catalog?**

A unified governance layer for managing data access, lineage, and metadata across Databricks workspaces.

8. **How do you create a table in Unity Catalog?**

```
CREATE TABLE catalog.schema.table_name (columns...) USING DELTA
```

9. **How do you use catalogs and schemas in Databricks?**

```
spark.sql("USE CATALOG finance_prod") spark.sql("USE SCHEMA loan_mgmt_db")
```

10. **What is a managed vs external table?**

- Managed: Databricks controls the storage location.
- External: You specify the storage path.

◆ Spark Execution & Optimization

3. **What is the Catalyst optimizer?**
Spark's query optimization engine for logical and physical plan generation.
4. **What is Tungsten in Spark?**
A Spark execution engine that optimizes memory and CPU usage.
5. **What is a DAG in Spark?**
Directed Acyclic Graph — represents the sequence of transformations and actions.
6. **What is a stage in Spark?**
A set of tasks that can be executed in parallel. Spark breaks jobs into stages.
7. **What is a task in Spark?**
The smallest unit of work in Spark — executed on a single partition.
8. **What is partitioning in Spark?**
Dividing data into chunks for parallel processing. Helps with performance and scalability.
9. **How do you repartition a DataFrame?**

```
df.repartition(10)
```

10. **How do you coalesce partitions?**

```
df.coalesce(2)
```

11. **What is broadcast join?**
A join where a small table is sent to all worker nodes to speed up join performance.
12. **How do you monitor Spark jobs in Databricks?**
Using the Spark UI, job run history, and cluster metrics dashboard.

Would you like me to continue with the 50 moderate-level questions next? Or generate a downloadable study guide with all 250 questions?

Section 2: Moderate Questions & Answers

◆ Transformation Logic

1. **How did you convert interest rates from basis points to percentages?**
I used `col("interest_rate_bps") / 100` to derive `interest_rate_pct` for better readability and downstream calculations.
2. **How did you calculate EMI in PySpark?**
Using the formula:

$$EMI = \frac{P \cdot r}{1 - (1 + r)^{-n}}$$

3. where r is monthly interest rate and n is tenure in months.
4. **How did you convert tenure from years to months?**
`df.withColumn("tenure_months", col("tenure_years") * 12)`

5. **How did you flag overdue loans?**
Compared `due_date` with `current_date()` and created a boolean column `is_overdue`.
6. **How did you normalize currency values?**
Multiplied `loan_amount_usd` by `exchange_rate` to derive `loan_amount_inr`.
7. **How did you classify loan types?**
Used `when(...contains("machinery")...)` logic to tag loans as "Working Capital", "Construction Loan", or "General".
8. **How did you assign risk scores?**
Based on credit score and default flag using conditional logic:
 - Score 1: High credit, no default
 - Score 2: Medium credit
 - Score 3: Low credit or default
9. **How did you aggregate exposure per borrower?**
Grouped by `borrower_id` and calculated `total_exposure`, `avg_interest_rate`, and `loan_count`.
10. **How did you parse date strings?**
Used `to_date("issue_date_str", "yyyy-MM-dd")` to convert to `DateTime`.
11. **How did you standardize all date columns?**
Iterated through columns with "date" in name and applied `to_date()`.

◆ Enrichment Logic

1. **How did you enrich loan data with borrower demographics?**
Joined `loan_df` with `borrower_profile_df` on `borrower_id`.
2. **How did you add credit scores?**
Joined with `credit_bureau_df` on `borrower_id`.
3. **How did you attach loan officer info?**
Joined with `loan_officers_df` on `loan_id`.
4. **How did you map branch location?**
Joined with `branch_master_df` on `branch_id`.
5. **How did you include repayment history?**
Aggregated `repayment_df` by `loan_id` to get `repayment_count` and `missed_payments`.
6. **How did you add collateral valuation?**
Joined with `collateral_df` on `collateral_id`.
7. **How did you flag fraud risk?**
Set `fraud_flag = True` if `missed_payments > 3`.
8. **How did you classify borrower sectors?**
Joined with `industry_df` on `industry_code`.
9. **How did you tag loan purpose?**
Used keyword matching on `loan_purpose` to derive `purpose_tag`.
10. **How did you mark compliance status?**
Flagged as "Compliant" if `principal > ₹5 Cr` and `collateral` exists.

◆ Schema Design & Validation

1. **How did you design the final schema?**
Combined key fields from all 10 tables into a unified schema with 30+ attributes.

2. **How did you handle schema drift in JSON files?**
Used `mergeSchema=True` and validated new fields before integrating.
3. **How did you enforce schema consistency across tables?**
Applied `StructType` definitions and column normalization during ingestion.
4. **How did you validate critical fields?**
Dropped rows missing `loan_id` or `borrower_id` using `dropna(subset=...)`.
5. **How did you handle duplicate records?**
Used `dropDuplicates()` on primary keys like `loan_id`.
6. **How did you track schema evolution?**
Maintained versioned schemas and used Delta Lake's `DESCRIBE HISTORY`.
7. **How did you ensure referential integrity in joins?**
Validated join keys, used left joins, and logged unmatched records.
8. **How did you handle nulls in enrichment joins?**
Used left joins and filled missing values with defaults or flags.
9. **How did you validate enrichment completeness?**
Checked join success rates and row counts before and after joins.
10. **How did you document schema lineage?**
Created schema diagrams and field-level lineage maps using markdown and `diagrams.net`.

◆ Orchestration & Delivery

1. **How did you automate the pipeline?**
Scheduled notebooks via Databricks Jobs with dynamic `biz_date` parameter.
2. **How did you pass parameters to notebooks?**
Used `dbutils.widgets.text("biz_date", "")` and `dbutils.widgets.get("biz_date")`.
3. **How did you deliver data to RiskOps?**
Wrote Parquet files to Azure File Storage, partitioned by `biz_date`.
4. **How did you deliver data to FinanceControl?**
Saved final DataFrame as a Delta table in Unity Catalog.
5. **How did you deliver data to AuditTrail?**
Exported CSV files to Azure, partitioned by `date`.
6. **How did you ensure delivery success?**
Logged file paths, row counts, and timestamps after each write.
7. **How did you handle delivery failures?**
Implemented retry logic and alerts via job failure notifications.
8. **How did you monitor job runs?**
Used Databricks job history, Spark UI, and cluster metrics.
9. **How did you validate final outputs?**
Compared row counts, schema checks, and sample data across delivery formats.
10. **How did you support incremental loads?**
Partitioned outputs by `biz_date` and used filters during reads.

◆ Cluster & Performance

1. **What cluster configuration did you use?**
Standard_DS3_v2 (Azure), 4 vCPUs, 14 GB RAM, 4–8 workers with auto-scaling.
2. **Why did you enable auto-scaling?**
To optimize resource usage during peak transformations and reduce idle costs.

3. **How did you handle skewed joins?**
Used salting techniques and broadcast joins for small dimensions.
4. **How did you optimize read/write performance?**
Used partitioning, caching, and coalescing during delivery.
5. **How did you manage cluster costs?**
Scheduled jobs during off-peak hours and used spot instances where possible.
6. **How did you handle large joins?**
Repartitioned on join keys and used broadcast joins when feasible.
7. **How did you manage memory pressure?**
Tuned Spark configs like `spark.sql.shuffle.partitions` and used `persist()` selectively.
8. **How did you ensure cluster stability?**
Monitored executor logs, memory usage, and job retries.
9. **How did you handle job retries?**
Configured retry policies in Databricks Jobs and added error handling in notebooks.
10. **How did you document cluster setup?**
Maintained cluster specs, configs, and usage patterns in project documentation.

Would you like me to continue with the 100 advanced questions next, or generate a downloadable guide with all 250 questions?

Section 3: Advanced Questions & Answers

These cover optimization, cluster tuning, CI/CD, security, and productionization.

1. How did you handle schema evolution in Delta Lake?

- Enabled `mergeSchema` during writes and used `ALTER TABLE ... ADD COLUMNS` for controlled evolution.

1. How did you monitor job performance in Databricks?

- Used Spark UI, Ganglia metrics, and Databricks job run history. Also integrated with Azure Monitor for alerts.

1. How do you optimize Spark joins in large datasets?

- Use broadcast joins for small dimensions, repartition on join keys, avoid skewed joins, and cache intermediate results.

2. What is predicate pushdown and how does it help?

- It pushes filters down to the data source, reducing the amount of data read into memory.

3. How do you handle skewed data in joins?

- Use salting, skew hints, or broadcast joins. Also, filter out high-frequency keys or aggregate before joining.

4. What is the difference between `repartition()` and `coalesce()`?

- `repartition()` increases/decreases partitions with a full shuffle. `coalesce()` reduces partitions without shuffle.

5. How do you tune `spark.sql.shuffle.partitions`?

- Reduce it from the default (200) to a lower number (e.g., 50) for small jobs to avoid unnecessary tasks.

6. How do you cache a `DataFrame` and when should you unpersist it?

- Use `df.cache()` or `df.persist()`. Unpersist when the data is no longer needed to free memory.

7. How do you optimize wide transformations like `groupBy` or `join`?

- Repartition on group/join keys, cache inputs, and avoid unnecessary shuffles.

8. What is the role of the Catalyst optimizer?

- It builds logical and physical plans to optimize query execution in Spark SQL.

9. How do you monitor Spark job performance?

- Use Spark UI, Ganglia, and Databricks job run history. Monitor stages, tasks, and shuffle read/write.

10. What is the impact of partition size on performance?

- Too small: overhead from too many tasks. Too large: memory pressure. Ideal size: 100–200 MB per partition.

11. How do you handle large file writes to Azure Blob from Spark?

- Use `coalesce()` to reduce small files and write in optimized Parquet format.

12. What is the difference between narrow and wide transformations?

- Narrow: data from one partition (e.g., `map`). Wide: data shuffled across partitions (e.g., `join`, `groupBy`).

13. How do you avoid small file problems in Delta Lake?

- Use `OPTIMIZE` command or `coalesce()` before writing.

14. How do you optimize Delta table reads?

- Partition by high-cardinality columns, use Z-Ordering, and leverage data skipping.

15. What is Z-Ordering in Delta Lake?

- It co-locates related data blocks to improve read performance on frequently filtered columns.

16. How do you handle large shuffles in Spark?

- Increase executor memory, tune shuffle partitions, and use `broadcast` joins where applicable.

17. How do you optimize Parquet writes?

- Use `snappy` compression, partitioning, and avoid writing too many small files.

18. How do you use `explain()` in Spark?

- To view logical and physical plans and understand execution strategies.

19. How do you reduce job startup time in Databricks?

- Use cluster pools or keep a lightweight interactive cluster warm.

20. How do you handle out-of-memory errors in Spark?

- Increase executor memory, reduce partition size, or cache less data.

21. What is the role of `spark.sql.autoBroadcastJoinThreshold`?

- Controls the size limit (in bytes) for automatic broadcast joins.

22. How do you optimize window functions?

- Partition by selective keys, avoid sorting large datasets, and cache inputs.

23. How do you handle large Delta merges?

- Use partition pruning, optimize before merge, and limit update scope.

24. How do you monitor shuffle size?

- Use Spark UI → Stages tab → Shuffle Read/Write metrics.

25. How do you reduce lineage size in Spark?

- Use `checkpoint()` to truncate lineage and avoid stack overflow in long pipelines.

Cluster Tuning & Management (Q26–Q40)

- 1. What instance type did you use in LendingFlow360?**
 - `Standard_DS3_v2` with 4 vCPUs, 14 GB RAM, and 4–8 workers (auto-scaling enabled).
- 2. Why did you choose auto-scaling?**
 - To optimize cost and performance — scale up during joins, scale down during idle time.
- 3. How do you configure cluster pools in Databricks?**
 - Define a pool with min/max nodes and attach jobs to reduce startup latency.
- 4. What is the difference between interactive and job clusters?**
 - Interactive: for development and notebooks. Job: ephemeral, used for scheduled jobs.
- 5. How do you handle cluster termination policies?**
 - Set idle timeout (e.g., 10 mins) to auto-terminate and reduce costs.
- 6. How do you monitor cluster health?**
 - Use Spark UI, Ganglia, and Databricks metrics dashboard.
- 7. How do you manage cluster libraries?**
 - Install via cluster UI or init scripts. Use `%pip` in notebooks for scoped installs.
- 8. How do you isolate environments across teams?**
 - Use separate clusters, Unity Catalog schemas, and cluster policies.
- 9. How do you handle dependency conflicts?**
 - Use virtual environments or pinned versions in `requirements.txt`.
- 10. How do you manage secrets in Databricks?**
 - Store in Databricks Secrets and access via `dbutils.secrets.get()`.
- 11. How do you secure cluster access?**
 - Use role-based access control (RBAC), cluster ACLs, and Unity Catalog permissions.
- 12. How do you configure logging in Databricks?**
 - Use `spark.log.level`, `log4j` properties, and export logs to Azure Monitor or S3.
- 13. How do you handle cluster restarts during job execution?**
 - Use job retries and idempotent logic in notebooks.
- 14. How do you manage cluster costs?**
 - Use spot instances, auto-scaling, and monitor usage patterns.
- 15. How do you enforce cluster policies?**
 - Define policies in admin console to restrict instance types, auto-scaling, and permissions.

Security, Governance & Compliance (Q41–Q60)

2. **How do you manage data access in Unity Catalog?**
 - Use fine-grained access control at catalog, schema, table, and column levels.
3. **How do you implement row-level security?**
 - Use dynamic views with `IS_MEMBER()` or `current_user()` filters.
4. **How do you audit data access in Databricks?**
 - Use Unity Catalog audit logs and Azure Activity Logs.
5. **How do you secure data in transit and at rest?**
 - Use HTTPS for transit and Azure-managed encryption for storage.
6. **How do you handle PII in LendingFlow360?**
 - Masked sensitive fields, encrypted storage, and restricted access via Unity Catalog.
7. **How do you implement data lineage?**
 - Unity Catalog tracks lineage automatically. Also documented lineage manually in diagrams.
8. **How do you manage schema evolution securely?**
 - Enabled `mergeSchema`, versioned schemas, and validated changes before applying.
9. **How do you ensure auditability of data?**
 - Used Delta time travel, versioning, and partitioned delivery for traceability.
10. **How do you handle GDPR/DSAR requests?**
 - Used Delta `DELETE` and `VACUUM` to remove user data and maintain compliance.
11. **How do you restrict access to production data?**
 - Used Unity Catalog roles, workspace separation, and cluster access controls.

...

Let me pause here and ask: would you like me to continue with the remaining 50 advanced questions (Q61–Q100) covering CI/CD, productionization, monitoring, and delivery automation? I can also compile this into a downloadable format if you'd like.

Section 4: Scenario-Based Questions & Answers

These test real-world problem-solving and decision-making.

1. **Scenario: A join between loan and repayment tables is taking too long. What do you do?**
 - Check for skewed keys, repartition on join keys, use broadcast join if one table is small, and cache intermediate results.
1. **Scenario: A downstream team reports missing data in their Parquet delivery. How do you debug?**

- Validate row counts at each stage, check partition filters, verify `biz_date`, and confirm write success in logs.
1. **Scenario:** A new column is added to the JSON schema. How do you handle it without breaking the pipeline?
 - Use `mergeSchema=True` in read/write, validate schema drift, and update downstream logic if needed.

◆ Data Ingestion & Validation

1. **Scenario:** A CSV file has inconsistent headers across partitions.
Solution: Use `header=True`, infer schema, and validate column consistency. Apply schema override if needed.
2. **Scenario:** A JSON file contains nested arrays and optional fields.
Solution: Use `explode()` for arrays, `get_json_object()` or `from_json()` for nested fields, and handle nulls gracefully.
3. **Scenario:** A source file is missing critical fields like `loan_id`.
Solution: Drop rows using `dropna(subset=["loan_id"])` and log the count for audit.
4. **Scenario:** A CSV file has mixed date formats.
Solution: Use `to_date()` with multiple formats or regex-based parsing logic.
5. **Scenario:** A file fails to load due to corrupt rows.
Solution: Use `badRecordsPath` to isolate corrupt rows and continue ingestion.

◆ Data Cleaning & Transformation

1. **Scenario:** You find duplicate `loan_id` entries.
Solution: Use `dropDuplicates(["loan_id"])` and investigate source system logic.
2. **Scenario:** Some numeric fields have outliers (e.g., negative principal).
Solution: Apply domain rules to filter or flag invalid values.
3. **Scenario:** You need to standardize all date columns.
Solution: Loop through columns with "date" in name and apply `to_date()`.
4. **Scenario:** You need to normalize column names across sources.
Solution: Use `df.columns` and `rename` to lowercase snake_case.
5. **Scenario:** You need to calculate EMI for loans.
Solution: Apply EMI formula using Spark expressions and handle zero-interest edge cases.

◆ Enrichment & Joins

1. **Scenario:** A join between loan and borrower tables drops rows.
Solution: Use left join, validate join keys, and log unmatched records.
2. **Scenario:** You need to enrich loans with repayment history.
Solution: Aggregate repayment table by `loan_id` and join with loan table.
3. **Scenario:** Collateral data is missing for some loans.
Solution: Use left join and set `collateral_flag = False` for missing entries.
4. **Scenario:** You need to tag loan purpose based on free-text.
Solution: Use `when(...contains(...))` logic or NLP keyword matching.

5. **Scenario:** You need to classify borrower sectors.
Solution: Join with industry master table using `industry_code`.

◆ Schema Design & Validation

1. **Scenario:** A new column is added to the JSON schema.
Solution: Use `mergeSchema=True` and validate downstream compatibility.
2. **Scenario:** You need to enforce schema consistency across environments.
Solution: Define `StructType` schemas and validate during ingestion.
3. **Scenario:** A downstream team reports missing fields.
Solution: Compare schema versions, check delivery logic, and reprocess if needed.
4. **Scenario:** You need to track schema evolution.
Solution: Use Delta Lake's `DESCRIBE HISTORY` and maintain schema registry.
5. **Scenario:** You need to validate referential integrity.
Solution: Check join key uniqueness and completeness before enrichment.

◆ Final Delivery & Partitioning

1. **Scenario:** RiskOps needs daily Parquet files partitioned by `biz_date`.
Solution: Write using `partitionBy("biz_date")` and validate path structure.
2. **Scenario:** FinanceControl needs a Delta table with schema evolution.
Solution: Use `mergeSchema=True` and `overwrite` mode with version tracking.
3. **Scenario:** AuditTrail needs CSV files for compliance.
Solution: Write using `.option("header", True).csv(path)` and partition by date.
4. **Scenario:** A delivery path is incorrect due to a typo in `biz_date`.
Solution: Validate `biz_date` format and use `datetime.today().strftime("%Y%m%d")` fallback.
5. **Scenario:** You need to support incremental loads.
Solution: Filter source data by `biz_date` and partition outputs accordingly.

◆ Job Scheduling & Automation

1. **Scenario:** You need to pass `biz_date` to a scheduled notebook.
Solution: Use `dbutils.widgets.text("biz_date", "")` and retrieve with `get()`.
2. **Scenario:** A job fails due to cluster termination.
Solution: Enable retries, use job clusters, and monitor idle timeout settings.
3. **Scenario:** You need to run the pipeline daily.
Solution: Schedule via Databricks Jobs with dynamic parameters and alerts.
4. **Scenario:** You need to validate job success.
Solution: Log row counts, delivery paths, and timestamps post-write.
5. **Scenario:** A job runs but delivers empty files.
Solution: Check filters, partition logic, and source data availability.

◆ Monitoring & Debugging

1. **Scenario:** A join takes too long to execute.
Solution: Check for skew, repartition on join keys, and use broadcast join if feasible.
2. **Scenario:** A notebook runs out of memory.
Solution: Reduce partition size, cache selectively, and increase executor memory.

3. **Scenario:** A delivery file is missing.
Solution: Check write path, partition filters, and job logs.
4. **Scenario:** A cluster is underutilized.
Solution: Tune `spark.sql.shuffle.partitions` and monitor task parallelism.
5. **Scenario:** A job runs slowly during peak hours.
Solution: Reschedule to off-peak, use cluster pools, or increase worker count.

◆ Governance & Compliance

1. **Scenario:** You need to mask PII fields.
Solution: Apply hashing or redaction logic before delivery.
2. **Scenario:** You need to audit data access.
Solution: Use Unity Catalog audit logs and workspace activity reports.
3. **Scenario:** A user requests deletion under GDPR.
Solution: Use Delta `DELETE` and `VACUUM` to remove records.
4. **Scenario:** You need to restrict access to production data.
Solution: Use Unity Catalog roles and workspace separation.
5. **Scenario:** You need to track lineage.
Solution: Use Unity Catalog lineage features and document manually.

◆ Architecture & Design Decisions

1. **Scenario:** You need to support multiple environments (dev, QA, prod).
Solution: Use config-driven architecture and separate schemas/catalogs.
2. **Scenario:** You need to onboard a new data source.
Solution: Define schema, validate sample data, and integrate into ingestion logic.
3. **Scenario:** You need to refactor notebooks for modularity.
Solution: Split into ingestion, cleaning, enrichment, and delivery notebooks.
4. **Scenario:** You need to track delivery status.
Solution: Build a metadata tracker with row counts, timestamps, and file paths.
5. **Scenario:** You need to support schema evolution without breaking jobs.
Solution: Use `mergeSchema`, validate changes, and update downstream logic.

◆ Stakeholder Collaboration

1. **Scenario:** RiskOps requests additional fields.
Solution: Validate availability, enrich final schema, and update delivery logic.
2. **Scenario:** Finance team needs historical snapshots.
Solution: Use Delta time travel or versioned partitions.
3. **Scenario:** Audit team needs traceability for a specific loan.
Solution: Query by `loan_id`, check delivery logs, and provide lineage.
4. **Scenario:** A stakeholder reports inconsistent data.
Solution: Compare source vs delivery, validate transformations, and reprocess if needed.
5. **Scenario:** You need to present the pipeline to leadership.
Solution: Prepare diagrams, one-pager summary, and highlight business impact.

Would you like me to compile all 250 questions into a downloadable study guide or presentation format next?

Topics Covered

- Spark & PySpark
- Delta Lake & ACID
- Unity Catalog
- Data ingestion (CSV, JSON)
- Cleaning, transformation, enrichment
- Schema design & validation
- Joins, aggregations, window functions
- Final delivery (Parquet, Delta, CSV)
- Cluster configuration & tuning
- Job scheduling & parameterization
- Monitoring & alerting
- Compliance & auditability
- Real-world debugging & optimization

Believe in your preparation, trust your journey, and let your confidence speak louder than your nerves. You've got this — Good Luck!